



**ADDIS ABABA SCIENCE AND TECHNOLOGY
UNIVERSITY
COLLEGE OF ENGINEERING**

**DEPARTMENT OF SOFTWARE ENGINEERING
SOFTWARE CONFIGURATION MANAGEMENT(SCM)
MINI PROJECT ASSIGNMENT**

Section B Group 7

**Haileab Tesfaye ETS0714/14
Ephrem Mandefro ETS0536/14
Haileyesus Asrat ETS0718/14
Henok Tademe ETS0775/14**

Submitted to: Inst. Yibrah G.(MSc.)
Submitted Date: Fri Jan 2/2026 G.C

Table of Contents

1. Introduction	1
Purpose	1
Scope	1
References	1
2. Criteria for Identification	2
Configuration Items (CIs)	2
CI Identification and Naming Conventions:	2
3. Limitations & Assumptions	2
Factors Affecting the Plan	2
Limitations:	2
Assumptions:	2
4. CM Responsibilities & Authorities	3
5. Project Organization	3
Structure and CM Interfaces	3
6. Applicable Policies & Procedures	4
Standards, Directives, Methods	4
7. Planned Activities	5
Schedules, Resources, Milestones	5
8. CMP Maintenance	5
Update and Review Mechanisms	5

1. Introduction

Purpose

The purpose of this Software Configuration Management Plan (SCMP) is to establish a structured framework for managing configurations, versions, and changes throughout the development of the Inventory Tracker Website. This group project focuses on applying core SCM principles to ensure traceability, collaboration, and integrity of all project artifacts. The emphasis is on demonstrating SCM processes using Git/GitHub, with minimal programming complexity, allowing the team to collaborate effectively on a shared repository.

Scope

The scope encompasses managing configuration items (CIs) for a simple web application, including:

- A login/authentication page (login.html).
- A main dashboard/list page for inventory items (dashboard.html).
- One core functional action (e.g., add/view/update inventory items via app.py).
- A simple JSON file as the database (items.json).

The application utilizes HTML/CSS/JS for the frontend (e.g., style.css for styling) and Flask (Python) for the backend. Full functionality is not required; the project is a prototype to illustrate SCM practices such as version control, branching, change requests, baselines, releases, and audits. Boundaries: No complex features like multi-user support, advanced authentication, or relational databases; limited to basic non-functional requirements such as usability and basic performance (e.g., quick loading of pages). The plan covers the entire group project lifecycle from initial setup on December 2, 2025, to final submission on December 30, 2025, with emphasis on team collaboration via GitHub.

Organizational Relationships

The project is managed collaboratively by the group under the Department of Software Engineering. Organizational relationships include:

- Team members interacting via GitHub for pulls, reviews, and issues, with weekly virtual meetings for coordination.
- Instructor for oversight, feedback, and resolution of disputes.
- No external organizations or stakeholders involved.

References

- IEEE Std 828-2012: IEEE Standard for Configuration Management in Systems and Software Engineering.
- Lecture 5 slide on SCMP
- GitHub Help (docs.github.com).
- Flask Documentation (flask.palletsprojects.com).
- Assignment Document

2. Criteria for Identification

Configuration Items (CIs)

Configuration Items (CIs) are project artifacts essential for development, documentation, or operation that require version control and change management. CIs are identified early and tracked throughout the project to support group collaboration.

Criteria for inclusion:

- Relevance to project functionality, structure, or SCM processes.
- Potential for changes or updates by team members.
- Documented in the CI Register for visibility and accountability.

CI Identification and Naming Conventions:

Specific CIs for this project include:

- Requirements document (e.g., Requirements.docx).
- UI mockups (e.g., DashboardMockup.png or sketches).
- Frontend files: login.html, dashboard.html, style.css.
- Backend file: app.py (Flask application).
- Database file: items.json, users.json
- README.md (project overview and setup instructions).
- SCMP document (this file).
- Change Request forms, Change Log, Baseline Records, and Audit Report.

Naming Conventions: Prefixes are used for categorization, followed by a descriptive name and version number (e.g., DOC-Requirements-v1.0.docx, CODE-LoginPage-v1.0.html, CODE-Style-v1.0.css, CODE-App-v1.0.py, DATA-Items-v1.0.json). This ensures consistency across team contributions. All CIs are stored in the GitHub repository under structured folders (/docs for documents, /src for code and data).

3. Limitations & Assumptions

Factors Affecting the Plan

Limitations:

- Time constraint: Project timeline from December 2, 2025, to December 30, 2025, limits extensive testing or feature expansions; focus on core SCM deliverables.
- Team size: Group of 4 students (below the 5-7 recommended), leading to role overlaps and potential delays in parallel tasks.
- Tools: Dependent on free GitHub features; no paid enterprise tools for advanced collaboration.
- Non-functional requirements: Basic usability (e.g., intuitive interface) and performance (e.g., fast response times for simple operations) are included but not rigorously tested; no security features beyond basic login simulation.
- Scope: Prototype only; no production deployment or handling of large datasets.

Assumptions:

- All team members have reliable access to Git/GitHub, Python/Flask environments, and communication tools (e.g., email or messaging for meetings).
- Changes will be coordinated through group discussions to minimize conflicts.
- Internet connectivity is stable for GitHub pushes/pulls.
- The JSON database suffices for simple inventory tracking without scalability issues.
- Group members will adhere to assigned roles and communicate promptly on issues.

4. CM Responsibilities & Authorities

Roles, Responsibilities, Authority Lines

To facilitate group collaboration, roles are assigned with clear responsibilities and authority:

SCM Manager & Lead Developer (Haileab Tesfaye ETS0714/14)

Oversees SCMP implementation, manages baselines and releases, leads coding efforts (e.g., app.py and frontend files). Authority: Approve merges, resolve conflicts, and tag baselines.

Tester & Change Controller (Ephrem Mandefro ETS0536/14)

Tests changes for basic functionality, processes Change Requests (CRs), maintains Change Log. Authority: Review and approve CRs, verify implementations before merges

Documenter & Auditor (Haileyesus Asrat ETS0718/14)

Prepares and updates documents (e.g., CI Register, reports), conducts configuration audits. Authority: Update documentation CIs, report audit findings for team review.

Reviewer & Developer (Henok Tademe ETS0775/14)

Reviews pull requests (PRs) and documents, assists in development (e.g., JSON handling, CSS styling). Authority: Provide feedback on PRs, commit to feature branches after initial development.

Authority lines: All changes require review by at least one other member; SCM Manager has final approval on critical actions like releases. Escalations (e.g., disputes) go to the group for discussion or to the instructor. Weekly check-ins ensure accountability.

5. Project Organization

Structure and CM Interfaces

The project follows a collaborative group structure with GitHub as the primary CM platform to enable real-time teamwork.

- Structure: Flat hierarchy with shared access to the repository. Folders: /docs (documents), /src (code like app.py, html/css/js, items.json), /tests (basic tests), /releases (artifacts). Group meetings (virtual, 30-60 minutes weekly) for planning and issue resolution.
- CM Interfaces: Git for local version control; GitHub for remote collaboration, issues, PRs, and branching. Integration with development tools like VS Code for editing and Flask for backend runtime.

6. Applicable Policies & Procedures

Standards, Directives, Methods

- Standards: Adhere to IEEE 828-2012 for SCM planning. Follow Git best practices, including descriptive commit messages and atomic changes for group visibility.
- Directives: No direct commits to main branch; all changes via feature branches and PRs with at least one review. Incorporate basic non-functional requirements (e.g., ensure pages load quickly, interface is user-friendly).
- Methods: Use Git Flow branching model (main for stable, feature/ for developments like feature/add-item). Change control: Group-submitted CRs reviewed in meetings. Baselines tagged in Git for snapshots.

Versioning Rules: Semantic versioning (major.minor.patch, e.g., v1.0.0 for initial, v1.1.0 after features). Major changes for breaking updates, minor for additions, patch for fixes.

Branching Model: Main branch remains protected; feature branches (e.g., feature/login-update, feature/json-fix) for individual or paired work, merged via PRs after group review.

Change Control Process:

1. Team member identifies change need during development or testing.
2. Submit CR form (template in /docs) via GitHub issue or shared doc, including ID, description, reason, impact, and proposed solution.
3. Group review in meeting; approve/reject by SCM Manager and Change Controller.
4. Implement in dedicated feature branch; test locally.
5. Create PR for merge; update Change Log with details (date, status, implementer).

Baseline Management: Baselines are group-agreed snapshots tagged in Git (e.g., git tag BL1 && git push origin BL1). Accompanied by a one-page Baseline Record doc listing CIs, versions, and team sign-off. BL1: Repository setup + initial documents + CI list. BL2: Working prototype + CR fixes.

Tools Used: Git/GitHub for version control and collaboration; VS Code or similar IDE for editing; Python/Flask for backend; web browsers for testing.

7. Planned Activities

Schedules, Resources, Milestones

The schedule spans from project start on December 2, 2025, to submission on December 30, 2025, with group milestones to ensure progress:

- December 2-5: Group kickoff meeting, repo setup, assign roles, draft SCMP and CI Register (Milestone: Initial commit and BL1 tag).
- December 6-12: Develop prototype (code files like login.html, app.py), create feature branches, submit initial CRs (Milestone: Prototype functional, first PRs merged).
- December 13-18: Implement CRs, test changes, create BL2, prepare releases v1.0 and v1.1 (Milestone: Releases uploaded to GitHub with notes).
- December 19-25: Conduct configuration audit (PCA and FCA), address findings, update docs (Milestone: Audit report complete).
- December 26-29: Final reviews, prepare short project report, group sign-off (Milestone: All deliverables ready).
- December 30: Submission.

Resources:

- Human: 4 team members (4-6 hours/week each), with shared responsibilities.
 - Hardware: Personal laptops/computers.
 - Software: Git/GitHub (free tier), VS Code (free), Python 3.x with Flask library.
- Milestones include group check-ins (e.g., Dec 5, 12, 18, 25) to review progress and resolve issues collaboratively.

8. CMP Maintenance

Update and Review Mechanisms

The SCMP will be reviewed during weekly group meetings or after significant events (e.g., milestone completions). Updates: Propose via CR process; discuss and approve as a group; revise document and commit to repo with version update (e.g., SCMP-v1.1). The SCM Manager tracks changes. Due to the short project duration, no long-term maintenance is planned, but the final version will be archived in the repo for reference.